Chapter 8. Testing Node

Testing in the real world is generally about finding what's wrong with something. When it comes to code, testing is about finding what's right. It's the guarantee that something is working.

Untested code is simply dangerous. Efficient and scalable code is well-tested. Testing is not only about making sure the code is doing what it is supposed to do; it's also about making sure the code continues to do what it is supposed to do after any changes in the code, its environment, and its use patterns. Testing is about writing high-quality code and catching potential problems as early as possible.

Regular testing of code keeps it healthy and makes maintaining it easier. It also increases the confidence of its maintainers to make changes. Making changes to untested code is a recipe for disaster. A new feature in module X might break other features in module Y. You can't keep the dependencies in your head. You can't test all the code manually every time there is a change. There is no way around it. You have to write code to test your code, and yes, your testing code might have problems too. Tests might introduce false negatives and false positives. That's why it's extremely important to get the tests right. That is what this chapter is all about.

Assertions and Runners

A test in Node is a set of *assertions*. To execute the tests, you need a test *runner*. Node has built-in modules for both. Here's a simple test for the `map` method on arrays:

```
import test from 'node:test';
import assert from 'node:assert/strict';

test('doubles items for a 2*e transformer', () => {
  // Arrange
  const inputArray = [1, 2, 3, 10];
  const expectedResult = [2, 4, 6, 20];

  // Act
  const actualResult = inputArray.map((e) => 2 * e);

  // Assert
  assert.deepEqual(actualResult, expectedResult);
});
```

It helps to think of tests with the *Arrange-Act-Assert pattern*:

Arrange

This section is the setup. It's where we prepare the environment and prerequisites of the test.

Act

This section is where we execute the method we want to test.

Assert

This section is where we confirm our expectations.

To execute this test, you just put it in a Node script and run it with the `node` command.

Figure 8-1 shows the output.

The `node:test` module has a few objects that can be used to organize your tests and label them. I used the `test` method in this example to give the test the name `doubles items for a 2*e transformer`. When the `node` command sees this method, the built-in runner carries out the assertions in it to produce this output, which lists all test names and whether they succeeded or failed, along with the time duration for each

and a diagnostic summary about all tests. As you can see from this example, the one test we did was OK.

```
4   test('doubles items for a 2*e transformer', () => {
5     // Arrange
6     const inputArray = [1, 2, 3, 10];
7     const expectedResult = [2, 4, 6, 20];
8
9     // Act
10    const actualResult = inputArray.map((e) => 2 * e);
11
12    // Assert
13    assert.deepEqual(actualResult, expectedResult);
14  });
15
```

```
TERMINAL                                                    bash + ⌄ ⊟ 🗑 ···

$ node map.test.js
✔ doubles items for a 2*e transformer (1.201458ms)
ℹ tests 1
ℹ suites 0
ℹ pass 1
ℹ fail 0
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 14.01825
$
```

Figure 8-1. A passing test

The `node:assert` module has a set of assertion functions that we can use to implement the *Assert* section of any test. In this example, I used the `deepEqual` method to make sure the actual mapped array is deeply equal to the expected array. The simple `assert.equal` method will not work here as the arrays are different objects. The `deepEqual` method checks the equality of the array items, which is what we want here.

NOTE

Note how I used `node:assert/strict` in this example and that's what you should always do. The legacy mode for this module (without the `/strict`) is based on the `==` operator (which performs type conversions of the operands before comparison) and should be avoided.

Instead of the simple `test` method, we can use the `describe` and `it` methods:

```
import { describe, it } from 'node:test';
import assert from 'node:assert/strict';

describe('The map method on arrays', () => {
  it('doubles items for a 2*e transformer', () => {
    const inputArray = [1, 2, 3, 10];
    const expectedResult = [2, 4, 6, 20];

    const actualResult = inputArray.map((e) => 2 * e);

    assert.deepEqual(actualResult, expectedResult);
  });
});
```

The `it` method is basically an alias to `test`, but I prefer it as a reminder to stay consistent on labeling tests by answering the question, “What does this test validate?” and finishing the sentence: “It __.”

The `describe` method gives us a way to group tests together and describe their purpose. We’ll usually have multiple tests written for a method like `map`. We use `describe` to group them. This affects the output of running the tests as well.

DEEP VERSUS SHALLOW EQUALITY

When we check `a === b`, that's a shallow equality check that's making sure both `a` and `b` refer to the same memory location.

Deep equality, on the other hand, is when two objects have the same structure and values, regardless of their memory location.

For example, consider the following two arrays:

```
const arrA = [1, 4, 16];
const arrB = [2-1, 2*2, 4**2];

assert.equal(arrA, arrB);      // AssertionError
assert.deepEqual(arrB, arrB);  // Ok
```

Since the two arrays are different objects, the `assert.equal` check will fail. However, since both arrays have three elements and each element in `arrA` equals the element in `arrB` that's in the same position, these two arrays are deeply equal, and the `assert.deepEqual` check will pass.

Let's see what a failed test run looks like. Replace `deepEqual` with `equal` in the map method test, and run it again. Figure 8-2 shows the output I get.

```
    const expectedResult = [2, 4, 6, 20];

    const actualResult = inputArray.map((e) => 2 * e);

    assert.equal(actualResult, expectedResult);
  });
});
```

```
TERMINAL                                                    bash

$ node map.test.js

▶ The map method on arrays (5.967083ms)
ℹ tests 1
ℹ suites 1
ℹ pass 0
ℹ fail 1
ℹ cancelled 0
ℹ skipped 0
ℹ todo 0
ℹ duration_ms 28.350917

✖ failing tests:

test at map.test.js:5:3
✖ doubles items for a 2*e transformer (4.334292ms)
  AssertionError [ERR_ASSERTION]: Values have same structure but are not reference-
equal:

  [
    2,
    4,
    6,
    20
  ]
```

Figure 8-2. A failing test

Since the expected array does not equal the actual array, using `assert.equal` fails here.

TIP

I used the `map` method as an example here, but we shouldn't really be testing standard library methods like that. They are already well-tested. We should be testing our own code, our functions, our modules, our dependencies, and our systems. To start on that, let's talk about the different types of tests.

Types of Tests

There are four major types of tests: *unit*, *functional*, *integration*, and *end-to-end*. Each has different scopes and purposes. Practically, some of these types overlap sometimes, and in certain cases, some tests can be catego-

rized under multiple types. However, I think learning the difference between these types makes you write better tests.

To understand the difference between these types, let's start with a code example and see how different tests can be written for it. Let's assume we have two modules to manage products and orders in a database. For simplicity, we'll use simple arrays to store the records.

In a *products.js* file, we have the following:

```
let products = [
  { id: 1, name: 'Phone', price: 600 },
  { id: 2, name: 'Laptop', price: 2000 },
  { id: 3, name: 'Headphone', price: 100 },
];

const addProduct = (product) => {
  products.push(product);
  return product;
};

const getProductById = (id) => {
  return products.find((product) => product.id === id);
};

export { addProduct, getProductById };
```

In an *orders.js* file, we have the following:

```
const orders = [];

const createOrder = (productId, quantity) => {
  const order = { productId, quantity, status: 'pending' };
  orders.push(order);
  return order;
};

const updateOrderStatus = (orderId, status) => {
  const order = orders.find((order) => order.id === orderId);
  if (order) {
```

```
    order.status = status;
  }
  return order;
};

export { createOrder, updateOrderStatus };
```

NOTE

I initialized the products array with some testing data. Testing data is usually seeded for the tests in a different phase—for example, a setup phase run before all the tests. For simplicity, I included it in the module directly.

Unit Tests

A *unit test* is written for a small part of the code (a *unit*). A unit could be a single function, a group of related functions, a module, a component, or anything else that can be tested on its own, in isolation. Usually, any external dependencies are faked with double objects to maintain testing a unit in isolation. More on that shortly.

An example of a unit test for these modules is one that tests a single function in either module. For example, we should test the `getProductById` against one of the known testing data products and against a product that does not exist:

```
import { describe, it } from 'node:test';
import assert from 'node:assert/strict';

import { getProductById } from './products.js';

describe('getProductById', () => {
  it('finds a product that exists', () => {
    const product = getProductById(2);
    assert.deepEqual(product, {
      id: 2,
      name: 'Laptop',
      price: 2000,
```

```
    });
  });

  it('returns undefined for a product that does not exist', () => {
    const product = getProductById(-1);
    assert.equal(product, undefined);
  });
});
```

NOTE

For the remaining examples, I'll omit the `import` statements for `node:test` and `node:assert` for brevity.

Functional Tests

A *functional test* is written for a *feature* of the code (a functionality). A feature could be a single unit or multiple units, but this type of testing is not run in isolation. Functional tests often interact with external dependencies like databases and networks.

An example of a functional test for the orders module is one that tests the functionality of placing an order and updating its status:

```
import { createOrder, updateOrderStatus } from './orders.js';

describe('Order Management', () => {
  it('places an order and updates its status', () => {
    const newOrder = createOrder(1, 1);
    const updatedOrder = updateOrderStatus(
      newOrder.id,
      'completed',
    );
    assert.equal(updatedOrder.status, 'completed');
  });
});
```

Integration Tests

An *integration test* is written to check that different modules or services work correctly together. While functional tests are testing the functionality of a single module or service, integration tests are used to check how multiple modules or services integrate with each other.

An example of an integration test for the products/orders modules is one that tests how order creation integrates with product retrieval:

```
import { getProductById } from './products.js';
import { createOrder } from './orders.js';

describe('Order Creation', () => {
  it('integrates with product retrieval', () => {
    const product = getProductById(1);
    const order = createOrder(product.id, 2);
    assert.equal(order.productId, product.id);
    assert.equal(order.quantity, 2);
  });
});
```

End-to-End Tests

An *end-to-end (e2e) test* is written to simulate a real-world usage of your code, from start to finish, across all components, modules, and services.

An example of an e2e test for the products/orders modules is one that tests the entire flow from product creation to order completion:

```
import { addProduct, getProductById } from './products.js';
import { createOrder, updateOrderStatus } from './orders.js';

describe('From product addition to order completion', () => {
  it('works', () => {
    addProduct({ id: 4, name: 'Tablet', price: 500 });
    const product = getProductById(4);
    const order = createOrder(product.id, 1);
    const finalOrder = updateOrderStatus(order.id, 'completed');
```

```
      assert.equal(product.name, 'Tablet');
      assert.equal(finalOrder.status, 'completed');
    });
  });
```

Knowing the different types of tests is good in general. We get to focus our testing strategy based on the needs of the application, and perhaps more importantly, when a test fails, its type is the first indicator of what went wrong.

However, there's often a thin line between these different types of tests. For example, to better unit-test the `getProductById` function, we should have the test add a product first, then try to find it. You might think of that as a kind of integration testing. The labels are not really that important. What's important is having a strategy that suits the application and organizing testing code according to that strategy. In an application with isolated modules that don't depend on each other at all, there's little place for integration tests. The focus in that application should be on unit tests. For other types of applications, the team might opt to not write unit tests at all and rely completely on integration tests and e2e tests.

Test Doubles

In many cases, certain behaviors need to be simulated for testing purposes. For example, when testing code that uses an external API, it's a bad idea to have your tests hit the actual API every time you run them. It's better to use a test double object to fake the response of the API and regularly maintain this test double based on actual responses from the API.

Test doubles are basically objects we use to simulate the behavior of real objects. Just like a stunt double in a movie performs actions in place of a real actor, a test double object simulates the action of a real object.

Test doubles are commonly used in unit tests, as these tests are usually focused on isolated units. For example, if you need to unit-test a function that first retrieves a record from a database, you can use a test double for the database retriever object and focus the test on the remaining logic of

that function. An integration or e2e test can take care of testing the actual retrieval of records from the database.

Using test doubles simplifies testing and improves the speed of running all tests. With the right test doubles, you don't need to set up a database, an API, or network channels to run simple tests in isolation.

There are a few different types of test doubles. To see them in action, let's start with a code example:

```
function notifyCompletion(taskId, { database, emails, mailer }) {
  const status = database.getStatus(taskId);

  if (status === 'complete') {
    const email = emails.notifications;
    mailer.sendEmail(email, `Task ${taskId} is complete.`);
  }
}
```

This is a simple function that has three dependencies: a `database` object, an `emails` object, and a `mailer` object. All three dependencies are injected into the function as arguments (following the dependency injection pattern).

Different types of test doubles can be used to simulate these services with varying features for testing.

Dummy objects are passed around but are never actually used. They are just placeholders. They do nothing at all.

For example, if we're not focusing on the `emails` service, we can use a dummy empty object in a test:

```
// Assert something
notifyCompletion(123, {
  database,
  emails: {},
```

```
  mailer,
});
```

Stub objects are used to replace a unit of code with one that always behaves in a fixed way. To use a stub for a function, we write a new function that will always return the same result.

For example, to stub the `database` service in `notifyCompletion`, we can do something like this:

```
const stubDatabase = {
  getStatus: function (taskId) {
    return 'complete';
  },
};

// Assert something
notifyCompletion(123, {
  database: stubDatabase,
  emails,
  mailer,
});
```

Mock objects are used to test interactions and flows between components. We can use them for cases when we want to make sure that a certain function is called with certain arguments, a function tries to send an email, or a set of methods are called in a specific order.

For the `notifyCompletion` case, we can mock the `mailer` object to make sure its `sendEmail` method is called with the right arguments:

```
const mockMailer = {
  sendEmail(email, message) {
    assert.equal(email, 'task@example.com');
    assert.equal(message, 'Task 123 is complete.');
  },
};

// Assert something
```

```
notifyCompletion(123, {
  database,
  emails,
  mailer: mockMailer,
});
```

Spy objects are used to gather information about function calls, such as how many times a function is called.

If we're interested in keeping track of how many times the `sendEmail` method is used, we can use the following spy object for it:

```
const spyMailer = {
  sendCount: 0,
  sendEmail (email, message) {
    this.sendCount++;
  },
};
```

Fake objects are basically a simplified implementation of a complex interface or class. They work like the real code but with some complexities removed.

For example, if we need to test both the `complete` and `incomplete` values for `getStatus`, we can use a fake `database` object like the following:

```
class FakeDatabase {
  constructor() {
    this.tasks = { 123: 'incomplete', 456: 'complete' };
  }

  getStatus(taskId) {
    return this.tasks[taskId];
  }
}
```

As you can see, choosing what test doubles to use completely depends on what you're testing and how much of it you want to test.

Test doubles don't have to be purely one of these types. We can combine them. We can make `FakeDatabase` spy on how many times `getStatus` is called and throw in some expectations to make it also a mock double.

The names and scopes of these types are good to know and understand, but don't be overwhelmed with them. Just remember that a test double object can be used to simulate the behavior of a real object, and it adds power to your test. You can use them to simplify your tests, keep track of calls, and verify expectations too.

Node has a built-in `mock` object within the `node:test` module that can be used to create test doubles and has some powerful features as well. Let's talk about a few examples.

You can use Node's `mock` object to mock a method on any object, including methods within built-in modules. For example, if you're testing a function that uses the `readFile` method from Node's `node:fs` module, you can use the `mock` object to skip the actual reading of files for testing purposes:

```
import fs from 'node:fs/promises';
import { mock } from 'node:test'

mock.method(fs, 'readFile', async () => 'Hello World');
```

Once a method is mocked, it gets a `.mock` property that can be used to track how and when that method is called. The following assertion makes sure the `readFile` method is called exactly one time:

```
assert.equal(
  fs.readFile.mock.calls.length,
  1,
);
```

The `mock` object can be used to mock timer functions like `setTimeout` and `setInterval`:

```
// Create a mock function
const fn = mock.fn();

// Mock the setTimeout API
mock.timers.enable({ apis: ['setTimeout'] });
// Now, any calls to setTimeout will not
// create a timer.

// Test it
setTimeout(fn, 500);
assert.equal(fn.mock.callCount(), 0);

// You can manually tick the mocked timer
mock.timers.tick(500);
assert.equal(fn.mock.callCount(), 1);
```

You can also use the `mock` object to mock date objects when you need to write a time-dependent test:

```
test('mocks the Date object', (context) => {
  // Mock the Date object
  context.mock.timers.enable({ apis: ['Date'] });

  // The initial date will be based on 0 in the UNIX epoch
  assert.equal(Date.now(), 0);

  // Advance in time will also advance the date
  context.mock.timers.tick(100);
  assert.equal(Date.now(), 100);
});
```

Note how in this example I used the test `context` argument. This is passed to each test function and can be used to interact with the test runner. It has many of the `node:test` module features attached to it for convenience. The `context.mock` object is one of them. You can use it directly without importing `mock` from `node:test`. Some of the context methods have advantages over their noncontext matching methods. For example, by using `context.mock`, the test runner will automatically restore all mocked functionality once the test finishes. This is not true for using the

`mock` object directly; in fact, with a direct `mock` object use, we'll need to manually reset things after we're done with them using a `mock.reset()` call.

There are many other methods attached to the context object. Here are a few examples:

`context.test`

This method can be used to structure your tests in a hierarchical manner by nesting subtests under a top-level test.

`context.diagnostic`

This method can be used to write a message to the test runner output. Any diagnostic information is included at the end of the test's results.

`.before`, `.after`, `.beforeEach`, *and* `.afterEach`

These methods can be used to execute code before/after all tests in a scope and before/after each test.

Here is an example to demonstrate:

```
test('top level test', async (t) => {
  t.afterEach((t) => t.diagnostic(`Finished running ${t.name}`));
  t.after((t) => t.diagnostic(`Finished running ${t.name}`));

  await t.test('subtest 1', (t) => {
    assert.equal(1, 1);
  });

  await t.test('subtest 2', (t) => {
    assert.equal(2, 2);
  });
});
```

Figure 8-3 shows the output.

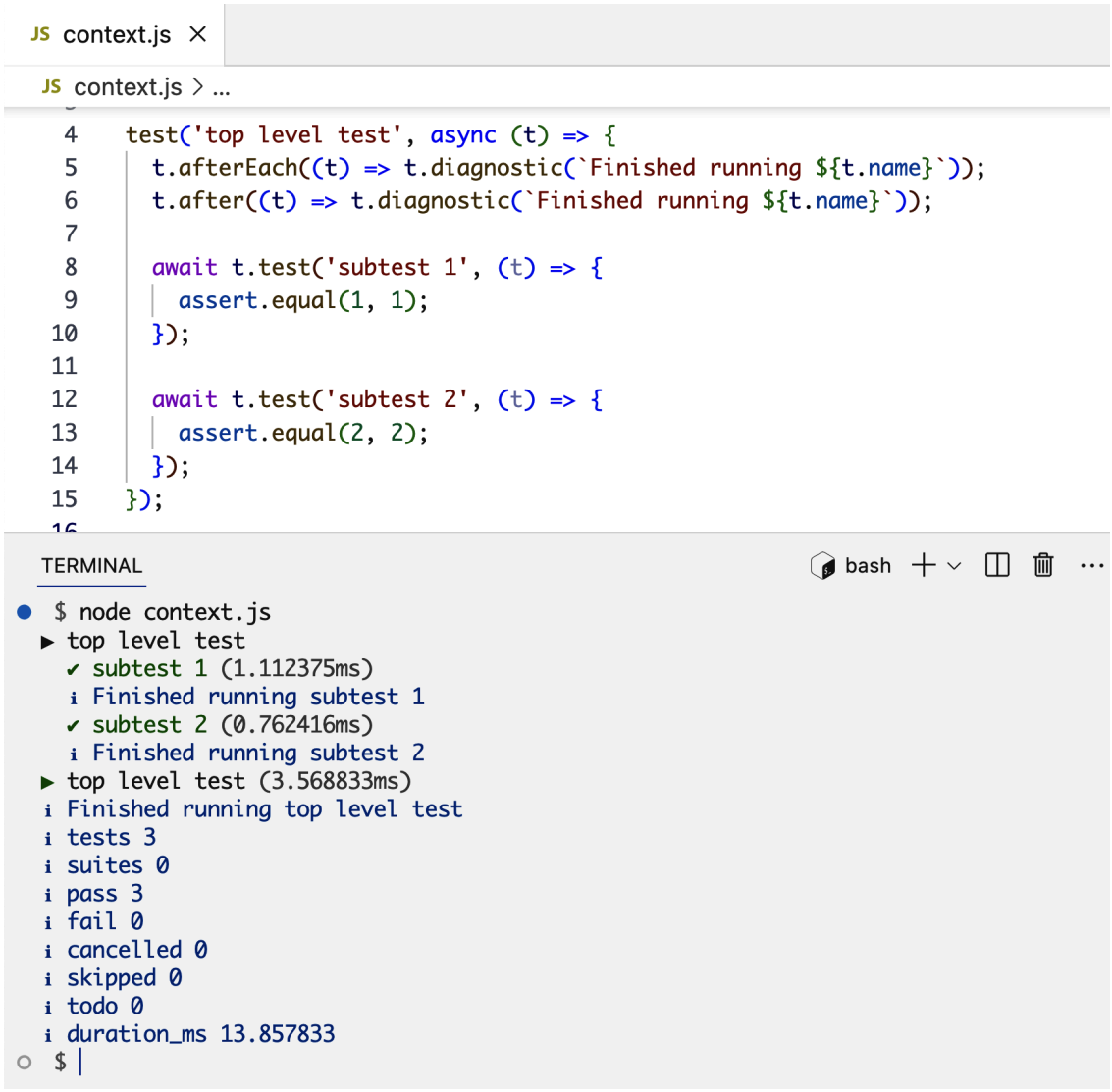

Figure 8-3. The example test with context

Note how `${t.name}` inside `afterEach` refers to the subtests, whereas it refers to the top-level test inside `after`.

The `after`, `before`, `afterEach`, and `beforeEach` functions are also available as imports from `node:test`.

Organizing and Filtering Tests

Where should test code be placed in an application? The two main strategies are to either place test files right next to the code files they test or isolate them in folders that can be mapped to the code they test.

For the first strategy, you put a test file right next to the module it's testing. If your module is *orders.js*, you create *orders.test.js* (or *ordersTest.js*,

or *orders.spec.js*) in the same folder as *orders.js*. Pick one of the naming formats, and stick with it throughout the entire application.

The main advantage of this approach is to quickly find the tests for any code you're dealing with and to quickly figure out when something isn't tested at all. I think this strategy is great for unit and functional tests.

For the other strategy, you mirror the structure of the application into a *tests* directory. For example, if you have *app/domain* and *app/models* folders, you would create *tests/domain* and *tests/models* directories.

The main advantage of this approach is to keep the code and tests separate while still being able to quickly find tests. However, this dependency on the application structure is not ideal. It ties the tests to any problems in the structure, and when a restructure is needed, a match of that restructure needs to happen in the *tests* folder as well, because otherwise it'll get harder to find tests.

A variant of this strategy is to maintain a *tests* folder within each *app* folder. You create *app/domain/tests* and *app/models/tests* folders and put all tests related to these modules in there. This still follows the app structure, maintains isolation between code and tests, and is easier to deal with when things get restructured.

No matter what approach you pick for your tests, consistency is the key. Pick a folder-naming convention and a file-naming convention as well, and stick with them for the entire application.

TIP

You can use code quality tools like ESLint and its plugins to flag any divergence from the naming conventions of a project.

Consistency in test organizations helps when you need to run a subset of tests. As you're working on one part of an application, you would usually frequently run the tests related to that part. You would still need to run all the tests, but you can do that less frequently.

Node supports a few ways for you to filter what tests to run. A second optional argument to the main methods of `node:test` can be used to skip a test altogether, mark it as a TODO test (which is executed but not included in failed tests), or mark a test as the only test that should be executed:

```
// The following test will be skipped
test('something', { skip: true }, () => {
  // ...
});

// The following test will be marked as TODO
test('something', { todo: true }, () => {
  // ...
});

// The following test will be the only executed test
// when the `node` command is run
// with a `--test-only` option
test('something', { only: true }, () => {
  // ...
});
```

You can also filter tests by their names. Node has two options for that: `--test-name-pattern="something"` and `--test-skip-pattern="something"`. This gives you the option of running or skipping tests when their names match certain patterns.

Combining all these features gives you a lot of flexibility in running and maintaining test files.

Test-Driven Development

Instead of writing tests for code, why not write code for tests? This is the test-driven development (TDD) approach. You start with a test, which will fail at first since there is no code. You write the code to make that test pass. Then you write another failing test, make it pass with more code, and so on. This creates a cycle of red-green (or fail-pass for tests). While

you're in a green state, you can refactor and improve your code, which is why this cycle is known as the red-green-refactor cycle.

The TDD inverted cycle might feel weird, but it's actually incredibly powerful. If you strictly follow a TDD approach, you will simply not have any code that's not tested. You can only write code to make a red test green, after all.

Not only that, I think TDD forces you to write better code. You don't waste time on unnecessary code. For any piece of code you're thinking of, you're forced to think of a test case for it. It gives you a clear understanding of the code and helps you make better decisions about how the code should evolve.

While TDD is simple to use, there are a few things to keep in mind. Let's look at an example. Let's say we want to create a function that validates email addresses. To use TDD, we might start with a test case like this:

```
describe('validateEmail', () => {
  it('works for a normal email', () => {
    assert(validateEmail('test@example.com'));
  });
});
```

Then we implement the simplest code that can make this test pass. This is important. Don't attempt to write more code than the absolute minimum of what can make this test pass. After you make the test pass, you can add another test case:

```
  it('works for less common emails', () => {
    assert(validateEmail('test.one@example.com.ab'));
    assert(validateEmail('test+one@1.com.ab'));
    assert(validateEmail('123@a-z.com.ab'));
  });
```

Write the code to make the test pass, then add another test:

```
  it('works for non-english characters', () => {
    assert(validateEmail('test@mañana.org'));
  });
```

Write the code to make the test pass, then add another test:

```
  it('fails for invalid addresses', () => {
    assert(validateEmail('test@') === false);
    assert(validateEmail('@test.com') === false);
  });
```

Note how the thinking is explicit here with tests, and every thought we have is documented with a test. For the last case, you would start thinking: should the function return false or should it throw an error? You make these decisions while designing the tests.

WARNING

TDD is great for unit tests and some functional tests, but it gets more challenging for bigger tests.

Continuous Integration

When working in a Node project that has tests, you should continuously run tests while making changes in your development environment. To make this easier, you can use the organizing and filtering techniques discussed earlier. You can also use the `--watch` option to automatically run tests every time there is a change.

This process is known as *continuous testing*. You should not push any code without making sure all tests pass locally first. But even tests that pass for you locally might not pass in another testing environment. This is why a tested project needs a continuous integration (CI) strategy as well.

With CI, you can use a platform like GitHub Actions or GitLab CI/CD to force a run for all tests after new changes are proposed to the project (for

example, in a pull request). Changes cannot be merged to the project unless the tests pass first. CI tools run all the tests in the cloud, and they can be integrated with your source control tools. For example, you can make them automatically block a pull request whose changes made the test suite fail.

CI leaves no room for common mistakes. Maybe someone did not run all the tests while making a change, or maybe the tests passed for them in their local development environment but would not pass in an environment closer to production. CI is the fail-safe approach.

Another subtle advantage of having a CI pipeline is that history can be preserved to track the progress of a suite of tests. You can track things like the time it takes for all the tests to run and the coverage of these tests, and you can also track how these things evolve.

Coverage is an indicator of how much of the code is tested. It's a good indicator for quantity, but it has nothing to do with quality. However, ensuring a good coverage amount is a good first step. You can set up your CI pipeline to generate a coverage report (the Node test runner has some support for that) and then demand that the percentage of code covered with tests not go below a certain threshold. If you push untested code, the coverage will decrease, and your code will not be merged.

Summary

While a small and simple application might do OK without tests, as the code evolves, a good testing strategy becomes the only way to grow an application with confidence.

Node has built-in tools for testing. There's the `node:test` module that provides methods to structure and run tests and the `node:assert` module that provides methods to perform assertions in tests.

There are four main types of tests: unit tests are used to test a single component in isolation, functional tests are used to test one functionality, integration tests are used to test the integration between multiple parts of

an application, and end-to-end tests are used to test a complete interaction with the application, from start to finish.

You can use test double objects (mocks, stubs, spies, etc.) to replace certain parts of the code for the purpose of isolating what to test and improving test quality and speed.

Test files can be collocated with the code they test or organized in a structure that mirrors the application structure. When running tests, you can have filters to run a subset of the tests and focus on only that subset.

In a well-tested Node application, you should integrate tests in your development workflow. You should run all tests locally before you push a change, or even better, write and run the tests before and after you make a change (which is known as test-driven development). You should also have the tests run in a staging environment through a continuous integration pipeline that gets automatically triggered for every new change that's proposed to the project, and use it to block that change if it makes any tests fail.

In the next chapter, we'll explore Node's cluster module and see how it can be used to manage running multiple Node processes.